

Zigbee-Mithörer

Vom Auspacken des USB-Sticks bis zum ersten mitgeschnittenen Funkpaket. Ein Handbuch für Menschen, die so etwas zum ersten Mal machen.

- 1 Auspacken und anstecken
- 2 Firmware aufspielen
- 3 Das erste Funkpaket lesen

Entwurf 2 — die Erweiterung läuft, das Buch wird noch gepflegt.

Es begleitet die Zigbee-Erweiterung und wird danach ein Kapitel im großen Handbuch. Alles, was noch nicht am Gerät bestätigt wurde, ist im Text ausdrücklich als **ungeprüft** gekennzeichnet — alles Übrige ist an einem laufenden Analyzer nachgemessen.

Inhalt

1	Wozu das gut ist — und was es nicht tut	
2	Was du brauchst	Einkaufs- und Prüfliste
3	Auspacken	Was in der Schachtel liegt
4	Zum ersten Mal anstecken	Erkennt der Rechner ihn?
5	Die Kennung festhalten	Fünf Minuten, die später Ärger sparen
6	Firmware aufspielen	der Analyzer macht es selbst
7	Das erste Funkpaket	Zuhören und verstehen
8	Was die Zahlen bedeuten	
9	Den richtigen Kanal finden	
10	Mit Wireshark ansehen	freiwillig
11	Wenn etwas nicht klappt	
12	Der Rückweg	ungeprüft — Notfall
13	Wörterbuch	
14	Im Analyzer: einschalten, ablesen, benennen	das eigentliche Ziel
15	Was noch fehlt	die künftigen Kapitel

SO LIEST DU DIESES BUCH

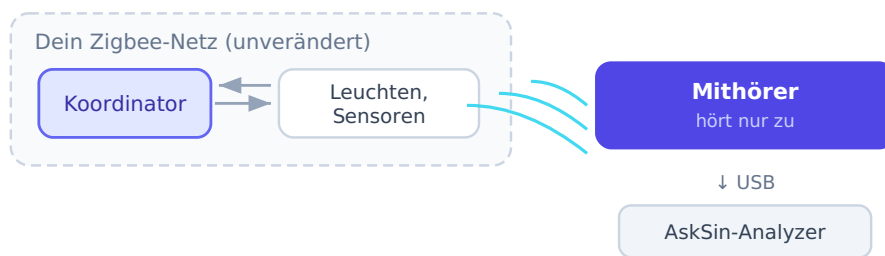
Die Kapitel 3 bis 7 sind eine **durchgehende Anleitung**. Arbeite sie der Reihe nach ab, ohne zu springen — jedes baut auf dem vorherigen auf. Kapitel 8 bis 13 sind zum Nachschlagen, **Kapitel 14 ist das eigentliche Ziel**: der Stick als Teil des Analyzers.

UNGEPRÜFT heißt: So steht es in den Unterlagen des Herstellers oder des Firmware-Projekts, wir haben es aber noch nicht selbst am Gerät gesehen. Weicht die Wirklichkeit ab, gilt die Wirklichkeit — und dieses Buch wird korrigiert.

1 Wozu das gut ist — und was es nicht tut

Dein Haus funkt auf zwei Bändern. Auf **868 MHz** reden die Homematic-Geräte miteinander; die hört der AskSin-Analyzer bereits mit und verrät dir, welches Gerät wie gut ankommt und wo es Funklöcher gibt. Auf **2,4 GHz** läuft dein Zigbee-Netz — Leuchten, Sensoren, Steckdosen. Dort hört der Analyzer bisher nichts.

Der USB-Stick, um den es in diesem Buch geht, ist das zweite Ohr. Er wird **kein Teil deines Zigbee-Netzes**. Er meldet sich nirgends an, er schaltet nichts, er antwortet nicht. Er sitzt daneben und schreibt mit, was durch die Luft geht — so wie ein Messgerät, das man an eine Leitung hält.



Der Mithörer steht neben dem Netz, nicht darin. Nichts an deiner Zigbee-Installation ändert sich.

Was du am Ende davon hast

- **Empfangsstärke je Gerät** — nicht nur „erreichbar“, sondern *mit wie viel Reserve*. Das kann dir dein Koordinator prinzipiell nicht sagen.
- **Ein Bild vom Störgeschehen** auf dem Kanal, auf dem dein Netz läuft.
- **Standortvergleich**, sobald mehrere Analyzer mithören: Welcher hört welches Gerät am besten, und welches Gerät hört überhaupt niemand.

Was es ausdrücklich nicht tut

Nicht	Warum
Geräte schalten oder anlernen	Der Analyzer ist ein Messgerät und bleibt passiv.
Inhalte von Nachrichten zeigen	Die Nutzdaten sind verschlüsselt. Sichtbar sind Absender, Empfänger, Art und Empfangsstärke — mehr brauchen wir nicht, und den Netzschlüssel wollen wir gar nicht haben.
Deinen vorhandenen Koordinator ersetzen	Der bleibt unangetastet. Ein Gerät kann nicht gleichzeitig Koordinator und Mithörer sein.
Mehrere Kanäle gleichzeitig hören	Ein Empfänger, ein Kanal. Wer springt, verpasst überall das meiste.

DAS WICHTIGSTE VORWEG

Du wirst in Kapitel 6 die Software auf dem Stick austauschen. **Danach ist er kein Zigbee-Koordinator mehr.** Nimm deshalb einen *neuen, zusätzlichen* Stick — **niemals den, an dem dein Zigbee-Netz hängt.** Steckst du den falschen um, steht dein ganzes Netz still.

Der Weg zurück steht in Kapitel 12. Er funktioniert — aber es ist ein Umweg, den man sich sparen kann.

2 Was du brauchst

2.1 Hardware

Teil	Wozu
SONOFF ZBDongle-E (Chip: EFR32MG21)	Der Mithörer. Achte auf das „E“ am Ende — die Variante „P“ hat einen anderen Chip, für den es keine fertige Mithör-Software gibt.
USB-Verlängerung, 0,5 bis 1 m	Bringt die Antenne weg vom Rechner. Ein Raspberry Pi ist selbst eine Störquelle auf 2,4 GHz — direkt am Gehäuse misst du zum Teil ihn statt deiner Geräte. Kein Luxus, sondern Teil des Messaufbaus.
(entfällt) Ein Rechner mit Chrome oder Edge	Brauchst du nicht mehr. Der Analyzer spielt die Software selbst auf (Kapitel 6). Nur wenn du den Umweg über den Browser gehen willst, brauchst du Chrome oder Edge — Firefox kann es nicht.
Der Raspberry Pi mit dem Analyzer	Dort hängt der Stick später dauerhaft. Pi 3, 4 oder 5 — alle drei gehen.

2.2 Was du wissen musst

Nichts über Funktechnik. Aber du solltest ein Fenster mit einer Befehlszeile öffnen können — auf dem Pi zum Beispiel über SSH — und einen Befehl abtippen. Jeder Befehl in diesem Buch steht vollständig da und wird erklärt.

2.3 Was du dir vorher notieren solltest

Ein Blatt Papier oder eine Textdatei. Du trägst im Lauf der Anleitung ein:

Angabe	Wo sie herkommt
Der Funkkanal deines Zigbee-Netzes	Kapitel 9 — bei den meisten ist es die 11, 15, 20 oder 25
Die Kennung des Sticks (Hersteller, Modell, Seriennummer)	Kapitel 5
Der Name, unter dem der Stick am Pi erscheint	Kapitel 5
Die Kurzadresse einer Lampe, die du zum Prüfen benutzt	Kapitel 9

ZEITBEDARF

Kapitel 3 bis 7 in einem Zug: **etwa eine Stunde**, wenn nichts klemmt. Der eigentliche Vorgang dauert Minuten — die Zeit geht für Nachsehen und Notieren drauf, und das ist gut angelegt.

3 Auspacken

In der Schachtel liegen der Stick, eine Antenne zum Aufschrauben und ein Zettel.

- 1 Antenne aufschrauben, falls eine dabei ist.**
Handfest, nicht mit Werkzeug. Der Anschluss ist klein und verträgt keine Gewalt. Ohne Antenne empfängt der Stick nur wenige Meter weit.
- 2 Antenne aufrecht stellen.**
Zigbee-Antennen strahlen quer zu ihrer Achse. Steht die Antenne senkrecht, hört sie in der Fläche am besten — also genau dort, wo deine Geräte sind. Liegt sie waagerecht, hörst du vor allem die Decke und den Boden.
- 3 Verlängerungskabel anstecken, Stick daran.**
Nicht direkt in den Pi. Siehe Kapitel 2.1 — das ist wirklich der Unterschied zwischen einer Messung und einem Zufallswert.
- 4 In den Raspberry Pi mit dem Analyzer stecken — über das Verlängerungskabel.**
Dort bleibt er. Das Aufspielen der Software erledigt der Analyzer selbst (Kapitel 6.1); ein zweiter Rechner ist dafür nicht mehr nötig. Nur wenn du den Umweg über den Browser gehen willst, steckst du ihn zunächst an einen Windows-Rechner.

ZWEI STICKS, DIE SICH ZUM VERWECHSELN ÄHNLICH SEHEN

Falls bei dir bereits ein Zigbee-Stick am Koordinator-Rechner steckt: Kleb dem neuen **jetzt sofort** ein Etikett auf. „Mithörer“ reicht. Nach dem Aufspielen der Software sehen die beiden gleich aus, verhalten sich aber völlig verschieden — und die Verwechslung merkst du erst, wenn dein Licht nicht mehr schaltet.

4 Zum ersten Mal anstecken

Ziel dieses Kapitels: **festzustellen, dass der Rechner den Stick sieht**. Mehr nicht. Wir spielen noch nichts auf.

Lies **4.1**, wenn der Stick — wie vorgesehen — am Raspberry Pi steckt. **4.2** brauchst du nur, wenn du den Umweg über einen Windows-Rechner und den Browser gehen willst.

4.1 Am Raspberry Pi

Öffne ein Befehlsfenster auf dem Pi und tippe *vor* dem Anstecken:

```
# Zeigt fortlaufend an, was der Rechner an seinen Anschlüssen bemerkt.  
sudo dmesg -w
```

Jetzt den Stick anstecken. Es sollten sofort mehrere Zeilen erscheinen. Interessant ist die mit `ttyUSB` oder `ttyACM` darin, zum Beispiel:

```
usb 1-1.3: New USB device found, idVendor=10c4, idProduct=ea60, bcdDevice= 1.00  
usbcore: registered new interface driver cp210x  
usbserial: USB Serial support registered for cp210x  
cp210x 1-1.3:1.0: cp210x converter detected  
usb 1-1.3: cp210x converter now attached to ttyUSB0
```

Die letzte Zeile ist die wichtige: **Der Stick heißt bei diesem Rechner gerade `ttyUSB0`**. Beenden mit `Strg + C`.

AM GERÄT ABGESCHRIEBEN

Diese Ausgabe stammt von einem laufenden Analyzer (Raspberry Pi 3), nicht aus einer Anleitung. Die Zahlen `10c4:ea60` und der Treibername `cp210x` gehören zum ZBDongle-E; siehst du sie, ist es der richtige Stick. Bei anderen Zigbee-Sticks stehen dort andere Werte — etwa `1a86` für einen CH34x-Baustein. **Die Nummer hinter `usb` (hier `1-1.3`) ist der Anschluss und bei dir eine andere**; darauf kommt es nicht an.

WARUM „GERADE“?

Diese Namen werden in der Reihenfolge vergeben, in der der Rechner die Geräte findet. Steckt morgen noch etwas anderes daran, kann aus `ttyUSB0` eine `ttyUSB1` werden — und dann zeigt jeder Befehl, den du dir notiert hast, auf das falsche Gerät. Deshalb bekommt der Stick einen festen eigenen Namen; dafür brauchen wir Kapitel 5.

4.2 Am Windows-Rechner

Rechtsklick auf das Startmenü → **Geräte-Manager** → Abschnitt **Anschlüsse (COM & LPT)**. Dort taucht beim Anstecken ein neuer Eintrag auf, etwa `COM4`. Diese Nummer brauchst du in Kapitel 6.

Erscheint nichts, fehlt der Treiber für den USB-Baustein. Im ZBDongle-E sitzt ein **CP2102N von Silicon Labs**; der passende Treiber heißt daher **CP210x**. Windows lädt ihn meist von selbst nach; wenn nicht, gibt es ihn bei Silicon Labs.

5 Die Kennung festhalten

Fünf Minuten, die dir später viel Sucherei ersparen. Wir notieren, *woran* der Rechner diesen einen Stick zweifelsfrei erkennt — unabhängig davon, in welchem Anschluss er steckt.

1 Hersteller- und Modellnummer auslesen

```
lsusb
```

Eine der Zeilen gehört zum Stick. Beim ZBDongle-E steht dort:

```
Bus 001 Device 004: ID 10c4:ea60 Silicon Labs CP210x UART Bridge
```

Der erste Wert (`10c4`) ist der Hersteller des USB-Bausteins, der zweite (`ea60`) das Modell. Notiere beides.

WARUM DER STICK EINE EIGENE NAMENSREGEL BEKOMMT

`10c4:ea60` ist ein CP2102N von Silicon Labs — und **derselbe Baustein wie auf der USB-Variante der Analyzer-Platine**. An diesen Nummern allein sind die beiden also nicht zu unterscheiden. Unterscheidbar macht sie erst der Text, den Itead hineinprogrammiert hat (`Itead / Sonoff Zigbee 3.0 USB Dongle Plus V2`). Genau darauf stützt sich die udev-Regel des Analyzers — ohne sie trüge der Stick den Namen des Sniffers.

2 Den sprechenden Namen auslesen

```
ls -l /dev/serial/by-id/
```

Ausgabe an einem laufenden Analyzer:

```
usb-Itead_Sonoff_Zigbee_3.0_USB_Dongle_Plus_V2_1a2b3c4d5e6f-if00-port0 -> ../../ttyUSB0
```

Der blaue Teil ist die **Seriennummer deines** Sticks und bei dir eine andere. Alles davor ist bei jedem ZBDongle-E gleich — und genau das machen wir uns gleich zunutze. Das ist ein **Name, der bleibt**: Er hängt nicht daran, in welchem Anschluss der Stick steckt.

Ab jetzt benutzen wir überall `$STICK` statt `ttyUSB0`. Diesen Befehl kannst du unverändert übernehmen:

```
# Nicht abtippen – ermitteln lassen. Der Name ist lang, und ein  
# Tippfehler darin sieht aus wie ein defekter Stick.  
STICK="$(ls -l /dev/serial/by-id/*Sonoff_Zigbee* 2>/dev/null | head -1)"  
echo "$STICK"
```

WARUM HIER NACH DEM NAMEN GESUCHT WIRD UND NICHT EINFACH DER ERSTE EINTRAG GENOMMEN

Am Analyzer hängt in aller Regel **mehr als ein serielles Gerät**. Wer blind den ersten Eintrag nimmt, erwischt irgendwann die Analyzer-Platine — und sucht den Fehler danach an der falschen Stelle. Der Filter `*Sonoff_Zigbee*` trifft den Stick und sonst nichts. Bleibt die Ausgabe leer, steckt er nicht oder wird nicht erkannt: zurück zu Kapitel 4.

3 Alles auf einmal, zum Abheften

```
udevadm info -a -n "$STICK" | grep -m5 -E 'idVendor|idProduct|serial|product'
```

Hier steht `"$STICK"` — der Wert aus dem Schritt davor; du musst nichts ersetzen. Diese Ausgabe ist die Grundlage für die feste Namensregel, die der Analyzer mitbringt. Schreib sie in deine Notizdatei.

MERKSATZ

`ttyUSB0` ist wie „das Auto auf dem dritten Parkplatz“ — der Name aus `by-id` ist das Kennzeichen. Für alles, was dauerhaft funktionieren soll, nimm das Kennzeichen.

DER ANALYZER HAT DAFÜR EINEN EIGENEN, KURZEN NAMEN

Im laufenden Betrieb benutzt der Analyzer weder `ttyUSB0` noch den langen `by-id` -Namen, sondern `/dev/asksin-zigbee`. Diesen Namen legt eine Regel des Betriebssystems an, die mit dem Analyzer ausgeliefert wird — du musst dafür nichts tun. Nach einem Update ist sie da.

Nachsehen kannst du so — die Ausgabe zeigt, worauf der kurze Name zeigt:

```
ls -l /dev/asksin-*
```

```
/dev/asksin-hat    -> ttyAMA0    # die Analyzer-Platine  
/dev/asksin-zigbee -> ttyUSB0    # der Zigbee-Stick
```

6 Firmware aufspielen

Jetzt tauschen wir die Software auf dem Stick aus. Bis hierher war alles umkehrbar; dieser Schritt ist es nur über den Umweg aus Kapitel 12.

Der kurze Weg steht in 6.1: Der Analyzer macht es selbst. Lies trotzdem den Kasten hier darunter — er gilt für jeden Weg. Die Abschnitte 6.2 bis 6.5 erklären, was dabei geschieht, und beschreiben die beiden Wege von Hand; die brauchst du nur, wenn 6.1 nicht durchkommt.

LETZTE KONTROLLE VOR DEM PUNKT OHNE EINFACHE UMKEHR

- Ist das der **neue, zusätzliche** Stick — nicht der, an dem dein Zigbee-Netz hängt?
- Ist der Koordinator-Rechner gerade **nicht** mit einem Stick verbunden, den du gleich verwechseln könntest?
- Klebt das Etikett „Mithörer“ drauf?

Dreimal ja? Dann weiter.

6.1 Der einfachste Weg: der Analyzer macht es selbst

Seit dem 18.08.2026 musst du für diesen Schritt **weder einen Browser noch einen zweiten Rechner noch die Konsole** bemühen. Der Analyzer holt die Software beim Urheber und spielt sie selbst auf. Es gibt zwei Stellen, an denen er das anbietet:

- **Beim Einrichten.** Wer im Einrichtungsassistenten Zigbee auswählt und den Stick schon stecken hat, wird gefragt: *„Auf dem Stick sitzt noch die Koordinator-Firmware — jetzt aufspielen?“*
- **Später, in der Weboberfläche.** *Einstellungen · Zigbee → Firmware auf dem Stick → „Sniffer-Firmware aufspielen ...“*

Beides fragt vorher nach, beides dauert wenige Minuten, und beides tut nichts, wenn die richtige Firmware schon drauf ist. Der Analyzer-Dienst startet dabei einmal neu — die Weboberfläche verliert also kurz die Verbindung. Das ist erwartet.

ZWEI STICKS? DANN VERWEIGERT DER ANALYZER DIE ARBEIT

Stecken **mehrere** SONOFF-Sticks an demselben Rechner, spielt er auf keinen davon etwas auf, und der Knopf bleibt gesperrt. Von außen ist nicht entscheidbar, welcher der Mithörer werden soll — und einer der beiden ist womöglich dein Koordinator. Zieh den anderen ab, dann geht es weiter. **Das ist Absicht und kein Fehler.**

WAS NOCH NICHT AM GERÄT BEWIESEN IST

Geprüft ist: Der Analyzer erkennt den Stick, erkennt eine bereits aufgespielte Mithörer-Firmware, richtet das Flash-Werkzeug ein und lädt die Firmware herunter (41 008 Byte).

UNGEPRÜFT ist bislang **der Schreibvorgang selbst** — dafür braucht es einen Stick, auf dem noch die Koordinator-Firmware sitzt, und der zweite war zum Zeitpunkt dieser Fassung noch unterwegs. Die Wege in 6.4 und 6.5 bleiben deshalb vollständig beschrieben.

6.2 Was aufgespielt wird

Eine Datei mit der Endung `.gbl` aus dem Projekt *Sniffer 802.15.4 SONOFF USB Dongle Plus E*. Sie macht aus dem Stick einen reinen Mithörer, der jedes empfangene Funkpaket als eine Zeile Text über den USB-Anschluss ausgibt.

Bezugsquelle: github.com/ErkSponge/Sniffer_802.15.4_SONOFF_USB_Dongle_Plus_E, Ordner `Output/Sniffer_802.15.4_SONOFF_USB_Dongle_Plus_E/`. Am Pi holst du sie direkt:

```
cd ~
curl -fL -o sniffer.gbl \
  https://raw.githubusercontent.com/ErkSponge/Sniffer_802.15.4_SONOFF_USB_Dongle_Plus_E/main/
  Output/Sniffer_802.15.4_SONOFF_USB_Dongle_Plus_E/Sniffer_802.15.4_SONOFF_USB_Dongle_Plus_E.gbl

# Kontrolle: rund 40 kB (am 18.08.2026: 41008 Byte). Sind es nur
# ein paar hundert Byte oder steht "Not Found" darin, hat sich der
# Pfad im Projekt geändert – dann ist es eine Fehlerseite, keine
# Firmware.
ls -l sniffer.gbl
```

WARUM WIR DIESE DATEI NICHT MITLIEFERN

Fremde Software hat eigene Nutzungsbedingungen. Wir verweisen darauf, statt sie in unser Projekt zu kopieren — so bekommst du immer die Fassung des Urhebers und nicht eine womöglich veraltete Kopie von uns.

6.3 Erst in den Bootloader — sonst wird das nichts

Im Stick stecken zwei Programme: die eigentliche Firmware und darunter ein kleines **Startprogramm (Bootloader)**, das allein neue Firmware schreiben kann. Ein Flashwerkzeug muss den Stick also erst dorthin bringen.

Der bequeme Weg ist, die laufende Firmware höflich zu bitten, in den Bootloader neu zu starten. Dazu muss das Werkzeug sie aber erst erkennen — und **genau daran scheitert der Web-Flasher beim ZBDongle-E regelmäßig**.

SO SIEHT DIESES SCHEITERN AUS

Der Browser meldet am Ende „Failed to probe running application type“. Im Protokoll steht davor eine lange Reihe von Versuchen — und mittendrin diese Zeile:

```
Received frame RStackFrame(version=2, reset_code=NcpResetCode.RESET_POWER_ON)
```

Das ist eine gültige Antwort deines Sticks. Sie beweist, dass USB, Treiber, Anschluss und Firmware in Ordnung sind — der Stick redet. Das Werkzeug kommt danach nur nicht weiter. **Ein Stick, der so antwortet, ist nicht defekt.**

6.4 Von Hand auf der Befehlszeile des Pi

Das Werkzeug heißt `universal-silabs-flasher`. Es kann den Stick über die Steuerleitungen der seriellen Schnittstelle **zwingen**, in den Bootloader zu starten — dafür braucht es die laufende Firmware gar nicht erst zu erkennen. Der Browser kann das nicht, weil er diese Leitungen nicht in der Hand hat.

```
# Einmalig einrichten, in einer eigenen Umgebung –
# so bleibt das Betriebssystem des Pi unberührt.
python3 -m venv ~/flasher
~/flasher/bin/pip install universal-silabs-flasher

# Was für Reset-Verfahren es gibt, sagt die Hilfe:
~/flasher/bin/universal-silabs-flasher --help

# Aufspielen.
# --bootloader-reset rts_dtr ist das Verfahren für Sonoff-Sticks.
# $STICK stammt aus Kapitel 5.
~/flasher/bin/universal-silabs-flasher \
  --device "$STICK" \
  --bootloader-reset rts_dtr \
  flash --firmware ~/sniffer.gbl
```

WORAN DU SIEHST, DASS DER BOOTLOADER GREIFT

Das Werkzeug meldet `Triggering rts_dtr bootloader`. Ab dieser Zeile ist der Stick in den Bootloader gezwungen — das Erkennen der laufenden Firmware, an dem der Browser scheitert, entfällt vollständig.

6.5 Der Weg im Browser — mit dem BOOT-Knopf

Wenn du am Windows-Rechner bleiben willst: Bring den Stick **von Hand** in den Bootloader. Dann ist das Erkennen der laufenden Firmware gar nicht nötig, und der Web-Flasher kommt sofort durch.

1 Gehäuse öffnen

— zwei kleine Kreuzschlitzschrauben auf der Seite des USB-Steckers, dann die Platine herauschieben. Neben dem USB-Stecker sitzen zwei winzige Knöpfe: **BOOT** und **RESET**.

- 2 **Stick abziehen**
— er muss stromlos sein.
- 3 **BOOT gedrückt halten und dabei einstecken**
— Knopf weiter halten.
- 4 **Im Browser „Connect“ anklicken**
und den Stick auswählen — er meldet sich jetzt als SiLabs-Gerät.
- 5 **Erst loslassen, wenn eine Bootloader-Version angezeigt wird.**
Das ist der Beweis, dass es geklappt hat.
- 6 **Die `.gbl`-Datei auswählen und den Vorgang starten**
— er dauert weniger als eine Minute.
- 7 **Nicht abziehen, solange etwas läuft.**
- 8 **Nach der Erfolgsmeldung: abziehen und wieder anstecken.**
Erst dadurch startet die neue Software.

DER GERÄTENAME BLEIBT — UND DAS HAT EINEN GRUND

Der Name `usb-Itead_Sonoff_Zigbee_3.0_...` stammt vom **USB-Baustein** (dem CP2102N), nicht von der Funk-Software, die wir gerade ausgetauscht haben. Deshalb heißt der Stick nach dem Aufspielen genauso wie vorher — am Gerät nachgesehen. Ermittle `$STICK` nach dem Wiederanstecken trotzdem neu: Es kostet nichts, und die Zuordnung zu `ttyUSB0` oder `ttyUSB1` kann sich sehr wohl verschieben.

WORAN DU MERKST, DASS ES GEKLAPPT HAT

Der Stick meldet sich nach dem Wiederanstecken erneut am Rechner (Kapitel 4 zeigt es dir). Der endgültige Beweis kommt aber erst im nächsten Kapitel: **Es kommen Textzeilen heraus.**

7 Das erste Funkpaket

Der spannendste Teil. Wir öffnen den Anschluss, sagen dem Stick, auf welchem Kanal er lauschen soll, und sehen zu.

7.1 Den Anschluss einstellen

Der Stick redet sehr schnell — **1 000 000 Zeichen je Sekunde**. Das muss man dem Betriebssystem einmal sagen:

```
# $STICK stammt aus Kapitel 5 – nach dem Aufspielen neu ermitteln!  
# raw    = nichts an den Zeichen verändern  
# -echo  = der Rechner plappert nicht zurück  
# -hupcl = der Stick startet NICHT neu, wenn der Anschluss  
#         geschlossen wird – sonst wäre der Kanal sofort wieder weg  
stty -F "$STICK" 1000000 raw -echo -hupcl
```

WARUM -HUPCL HIER ENTSCHEIDEND IST

Bei den meisten USB-Anschlüssen wird der Stick **neu gestartet**, sobald das letzte Programm den Anschluss schließt. Wer den Kanal mit einem Befehl setzt und mit einem zweiten zuhört, redet also mit einem Gerät, das zwischendurch neu gestartet ist — und wundert sich, dass nichts kommt. `-hupcl` verhindert das.

7.2 Kanal einstellen und zuhören

Zwei Befehle. Der erste sagt dem Stick den Kanal, der zweite zeigt, was hereinkommt. Setze deinen eigenen Kanal ein — wie du ihn herausfindest, steht in Kapitel 9.

```
# EINE offene Verbindung für Senden und Empfangen.
# exec 3<> hält den Anschluss offen, bis wir ihn selbst schließen.
exec 3<>"$STICK"

# Kanal einstellen
printf '{"C":11}\n' >&3

# Zuhören. Beenden mit Strg+C
cat <&3

# Wenn du fertig bist: Verbindung schließen
exec 3>&-
```

Nach Sekunden bis wenigen Minuten sollte so etwas erscheinen:

```
{ "L":50,"Q":255,"R":-94,"C":11,"S":"4188a31e48ffff00000912fcff000001cc0885da..."}
{"L":28,"Q":248,"R":-61,"C":11,"S":"6188c41e48b4a2f1e30028..."}
{"L":12,"Q":255,"R":-47,"C":11,"S":"0200a3..."}

```

WENN DU DAS SIEHST, IST DER SCHWIERIGE TEIL GESCHAFFT

Jede Zeile ist ein echtes Funkpaket aus deiner Wohnung, in dem Moment empfangen, in dem es gesendet wurde. Ab hier geht es nur noch ums Auswerten.

7.3 Nichts zu sehen?

Ruhe bewahren — auf einem stillen Kanal kann eine Minute vergehen, ohne dass irgendetwas funkt. Erzwinge Verkehr, indem du eine Zigbee-Lampe ein- und ausschaltest. Bleibt es dann immer noch still, weiter zu Kapitel 11.

7.4 Einen Mitschnitt aufheben

Für die spätere Entwicklung ist ein echter Mitschnitt Gold wert — an ihm wird der Decoder geprüft. Zehn Minuten reichen:

```
exec 3<>"$STICK"
printf '{"C":11}\n' >&3
timeout 600 cat <&3 > ~/zigbee-mitschnitt-$(date +%Y%m%d-%H%M).jsonl
exec 3>&-

wc -l ~/zigbee-mitschnitt-*.jsonl
```

Die letzte Zeile sagt dir, wie viele Pakete zusammengekommen sind.

8 Was die Zahlen bedeuten

Jede Zeile hat fünf Felder. Sie sind kurz benannt, weil es sehr viele Zeilen werden.

```
{ "L":50 , "Q":255 , "R":-94 , "C":11 , "S":"4188a31e..." }
```

- L** Länge des Pakets in Byte
- Q** Verbindungsgüte (LQI) — 0 bis 255, größer ist besser
- R** Empfangsstärke (RSSI) in dBm — näher an null ist besser
- C** Kanal, auf dem empfangen wurde
- S** das Paket selbst — Inhalt verschlüsselt, Adressen sichtbar

Eine Zeile, fünf Angaben. Nur zwei davon musst du wirklich verstehen.

8.1 R — die Empfangsstärke

Angegeben in **dBm**, immer negativ. **Näher an null ist besser**. –40 ist ein Gerät im selben Raum, –90 ist ein Gerät am Rand der Reichweite.

Die folgende Einteilung ist nicht aus einem Lehrbuch abgeschrieben, sondern **an 38 Geräten über 24 Stunden gemessen** — an einem Analyzer, der genau das tut, was dieses Buch beschreibt. Daneben steht die Verbindungsgüte, die sich bei diesen Werten tatsächlich einstellte:

Wert	gemessenes Q	Bedeutung
–26 bis –62	255 (voll)	Sehr kräftig. Gerät ist nah.
–63 bis –79	237 bis 254	Gut. So sieht der Normalfall aus.
–80 bis –86	149 bis 237	Schwächer, aber tragfähig.
–87	108	Die Kante. Ab hier bricht die Güte weg.
–88 und –89	8 bis 60	Grenzbereich. Pakete gehen verloren, sobald jemand anders funkt.
–90 und darunter	0 bis 10	Praktisch nichts mehr. Das Gerät ist noch zu hören, aber nicht mehr zuverlässig.

DIE KANTE BEI –87 DBM IST DAS BEMERKENSWERTE DARAN

Zwischen –86 und –90 dBm liegen vier Dezibel — die Verbindungsgüte fällt dabei von 149 auf 0. **Es gibt also keinen sanften Übergang**, sondern eine Kante, und ein Gerät kann sie mit einem verschobenen Möbel überschreiten. Genau deshalb genügt „erreichbar“ als Auskunft nicht: Ein Gerät bei –85 dBm und eines bei –89 dBm sind beide „erreichbar“ und trotzdem in völlig verschiedener Lage.

Genau diese Zahl ist der Grund für das ganze Vorhaben. Dein Koordinator sagt dir „Gerät erreichbar“ — er sagt dir nicht, ob es mit -52 oder mit -89 hereinkommt. Der Unterschied entscheidet darüber, ob dein Licht im nächsten Sommer noch zuverlässig schaltet.

8.2 Q — die Verbindungsgüte

Ein Wert von 0 bis 255, **größer ist besser**. Er beschreibt, wie sauber das Signal ankam — nicht wie laut. Ein starkes, aber verzerrtes Signal (etwa neben einem WLAN-Sender) hat hohes R und niedriges Q. Beide zusammen ergeben erst das Bild.

8.3 C, L und S

- **C** — der Kanal, auf dem empfangen wurde. Praktisch zur Kontrolle, dass die Einstellung angekommen ist.
- **L** — Länge des Pakets in Byte. **Genau 5** heißt: Empfangsbestätigung. Mehr passt in eine solche auch gar nicht hinein — zwei Byte Kopf, eine laufende Nummer, zwei Byte Prüfsumme. Sie machen einen erheblichen Teil des Verkehrs aus; der Analyzer zählt sie deshalb, speichert sie aber nicht einzeln.
- **S** — das Paket selbst in Hexadezimalschreibweise. Darin stecken Absender und Empfänger; die eigentliche Nachricht ist verschlüsselt und bleibt es.

WARUM VERSCHLÜSSELTE INHALTE KEIN VERLUST SIND

Wir wollen wissen, *ob* und *wie gut* Geräte gehört werden — nicht, was sie sagen. Adressen und Empfangsstärke reichen dafür vollständig. Den Netzschlüssel deines Zigbee-Netzes braucht der Analyzer nicht, und er soll ihn auch gar nicht bekommen.

9 Den richtigen Kanal finden

Zigbee benutzt die Kanäle 11 bis 26. Dein Netz läuft auf genau einem davon. Hörst du auf dem falschen, bleibt es still — der häufigste Grund für „es funktioniert nicht“.

9.1 In der Verwaltungsoberfläche nachsehen

Bei **Phoscon/deCONZ**: *Einstellungen* → *Gateway* → *Erweitert*. Dort steht der Kanal. Bei **zigbee2mqtt**: *Einstellungen* → *Netzwerk*. Bei **ZHA** in Home Assistant: *Einstellungen* → *Geräte* → *Zigbee Home Automation* → *Netzwerk*.

9.2 Wenn du gar nicht weiterweißt: alle absuchen

Der Stick kann jeden Kanal. Also probierst du sie durch — je zwanzig Sekunden, und wo Zeilen kommen, ist dein Netz:

```
# Einmal vorher, wie in 7.1:
stty -F "$STICK" 1000000 raw -echo -hupcl

# EINE offene Verbindung für den ganzen Durchlauf – wie in 7.2.
# Würde man je Kanal neu öffnen und schließen, startete der Stick
# zwischendurch neu und die Zählung wäre wertlos.
exec 3<>"$STICK"
for K in 11 15 20 25 12 13 14 16 17 18 19 21 22 23 24 26; do
    printf '{"C":%d}\n' "$K" >&3
    N=$(timeout 20 cat <&3 | wc -l)
    printf 'Kanal %2d: %4d Pakete\n' "$K" "$N"
done
exec 3>&-
```

Die vier zuerst genannten Kanäle sind die verbreitetsten — deshalb stehen sie vorn. Ein Durchlauf über alle sechzehn dauert gut fünf Minuten.

EIN HOHER WERT HEISST NICHT „MEIN NETZ“

Auf einem Kanal können auch die Zigbee-Geräte der Nachbarn funken — und WLAN stört obendrein. Der Durchlauf zeigt dir, *wo etwas los ist*, nicht, *wo dein Netz* ist. Diesen Beweis liefert erst 9.3.

9.3 Ein Gerät zum Wiedererkennen

Für die Abnahme brauchst du ein Gerät, das du sicher zuordnen kannst. Nimm eine Lampe, deren **Kurzadresse** du in der Verwaltungsoberfläche nachsehen kannst (vier Hexadezimalzeichen, etwa `0x8f2a`). Schalte sie ein und aus und sieh nach, ob in den mitgeschnittenen Zeilen genau in diesen Sekunden Verkehr auftaucht.

DAS IST DIE EIGENTLICHE ABNAHME

„Es kommen Zeilen" heißt nur, dass der Stick lebt. Erst **„ich schalte die Lampe im Flur, und genau dann tauchen Pakete auf"** beweist, dass du dein eigenes Netz hörst und nicht das des Nachbarn.

10 Mit Wireshark ansehen (freiwillig)

Für den Analyzer nicht nötig — er liest die Zeilen selbst. Aber wenn du einmal sehen willst, was in einem Paket steckt, ist Wireshark das Werkzeug dafür: Es zeigt Absender, Empfänger und Pakettyp im Klartext statt als Hexadezimalskette.

1 Wireshark installieren

— auf dem Pi mit `sudo apt install wireshark`, unter Windows vom Hersteller.

2 Das Zusatzprogramm aus dem Firmware-Projekt holen

— es übersetzt die Textzeilen des Sticks in das Format, das Wireshark erwartet. Im Ordner `Wireshark/` des Projekts: für Windows `Extcap_802.15.4_v1.0.2.exe`, für Linux `zig-bee_dongle_json_sniffer`.

3 In den Ordner „extcap“ kopieren

— wo der liegt, verrät Wireshark unter *Hilfe* → *Über Wireshark* → *Ordner*.

4 Wireshark neu starten

— der Stick erscheint jetzt in der Liste der Aufnahmequellen.

UNGEPRÜFT — diesen Weg haben wir nicht gebraucht und deshalb auch nicht durchgespielt. Der Analyzer wertet die Zeilen selbst aus; Wireshark ist nur für den Blick ins einzelne Paket.

11 Wenn etwas nicht klappt

Beobachtung	Wahrscheinliche Ursache und was hilft
Beim Anstecken passiert nichts (kein neues Gerät)	Anderen USB-Anschluss und anderes Kabel probieren — Ladekabel ohne Datenadern sind ein Klassiker. Am Pi: Verteiler ohne eigene Stromversorgung meiden.
<code>FileNotFoundError:</code> <code>No such file or directory</code>	Der Geräteiname stimmt nicht — meist wurde ein Platzhalter aus einer Anleitung mitkopiert. Namen nie abtippen , sondern mit dem Befehl aus Kapitel 5 ermitteln lassen. Kontrolle: <code>echo "\$STICK"</code> muss einen Pfad ausgeben, der wirklich existiert (<code>ls -l "\$STICK"</code>).
Web-Flasher meldet „Failed to probe running application type“	Der häufigste Fall — und kein Defekt. Das Werkzeug bekommt die laufende Firmware nicht erkannt und kommt deshalb nicht in den Bootloader. Steht im Protokoll irgendwo <code>RSTACK</code> , hat dein Stick sogar geantwortet. Abhilfe: den Analyzer aufspielen lassen (Kapitel 6.1) — er umgeht das Problem vollständig. Sonst Befehlszeile mit <code>--bootloader-reset</code> (Kapitel 6.4) oder BOOT-Knopf (Kapitel 6.5).
Windows: Anschluss da, aber unzuverlässig	Bei neueren Ausführungen mit CP2102-Baustein macht der <i>aktuelle</i> CP210x-Treiber Ärger; berichtet wird von der älteren Fassung 6.7.0.0 als Abhilfe. Betrifft nur den Weg über Windows — am Pi (6.1 und 6.4) stellt sich die Frage nicht.
Der Browser findet den Stick nicht	Firefox kann das nicht. Chrome oder Edge nehmen. Außerdem: Läuft nebenher noch ein Programm, das den Anschluss belegt (Wireshark, ein offenes <code>cat</code>)? Erst schließen.
Nach dem Aufspielen kommen keine Zeilen	Meist die Geschwindigkeit. Der <code>stty</code> -Befehl aus 7.1 muss <i>vor</i> dem Zuhören gelaufen sein, und zwar mit <code>1000000</code> .
Es kommt wirres Zeug statt lesbarer Zeilen	Ebenfalls die Geschwindigkeit — sie stimmt fast, aber nicht genau. <code>stty</code> wiederholen und darauf achten, dass keine anderen Einstellungen dazwischenfunken.
Lesbare Zeilen, aber sehr wenige	Falscher Kanal (Kapitel 9) oder die Antenne fehlt. Prüfen kannst du es, indem du eine Lampe schaltest.
Alle Empfangswerte sind schlecht (unter –85)	Antenne liegt waagrecht, steckt direkt am Pi-Gehäuse oder hinter Metall. Verlängerung benutzen, Antenne aufrecht stellen, mindestens eine Handbreit Abstand zu allem Metallischen.
Zeilen brechen mitten im Satz ab	Der Rechner kommt bei 1 MBaud nicht mit. Auf einem Pi 3 möglich. Kein Beinbruch — der Analyzer zählt später verworfene Pakete und zeigt sie an, statt sie zu verschweigen.
Dein Zigbee-Netz ist ausgefallen	Dann hast du den falschen Stick umgespielt. Kapitel 12 führt zurück. Ärgerlich, aber reparabel.

VORGEHEN, WENN NICHTS DAVON PASST

Ändere **immer nur eine Sache** und sieh dann nach. Zwei Änderungen auf einmal — anderer Anschluss *und* anderer Kanal — und du weißt hinterher nicht, welche geholfen hat. Notiere, was du probiert hast. Diese Notizen sind später mehr wert, als du jetzt glaubst.

12 Der Rückweg

Aus dem Mithörer wird wieder ein ganz normaler Zigbee-Koordinator. Nötig, wenn du den falschen Stick erwischst hast oder das Vorhaben beendest.

UNGEPRÜFT — dieser Weg ist nachvollziehbar beschrieben, wir mussten ihn aber nie gehen und haben ihn deshalb nicht selbst durchgespielt. Er benutzt dieselben Werkzeuge wie Kapitel 6; was dort funktioniert, sollte auch hier funktionieren. **Verlass dich trotzdem nicht darauf, sondern vermeide den Fall** — die Warnung in Kapitel 6 steht genau dafür da.

1 Koordinator-Software besorgen

Beim Hersteller: github.com/itead/Sonoff_Zigbee_Dongle_Firmware, Ordner `Dongle-E`.
Nimm die Fassung, die zu deiner Steuerung passt — deCONZ, zigbee2mqtt und ZHA erwarten teils verschiedene.

2 Genauso aufspielen wie in Kapitel 6

Derselbe Weg, dieselben Werkzeuge, nur eine andere Datei.

3 Netz wiederherstellen

Ist es der Stick deines echten Netzes gewesen, brauchst du zusätzlich die Sicherung deiner Netzdaten — sonst müssen alle Geräte neu angelernt werden. **Genau deshalb steht die Warnung in Kapitel 6.**

13 Wörterbuch

Zigbee

Funkstandard für Hausautomation auf 2,4 GHz. Geräte reichen Nachrichten untereinander weiter, sodass das Netz größer wird als die Reichweite eines einzelnen Geräts.

Koordinator

Das eine Gerät, das ein Zigbee-Netz aufspannt und verwaltet — bei dir der Stick am Steuerungsrechner. Es gibt genau einen je Netz.

Mithörer (Sniffer)

Ein Empfänger, der zu keinem Netz gehört und alles mitschreibt, was er hört. Er sendet nie. Das ist der Stick aus diesem Buch.

Kanal

Ein Frequenzabschnitt. Zigbee hat sechzehn davon, nummeriert von 11 bis 26. Ein Netz benutzt einen; ein Empfänger hört auf einem.

RSSI (Feld R)

Empfangsstärke in dBm, immer negativ. Näher an null ist besser. Siehe Kapitel 8.1.

LQI (Feld Q)

Verbindungsgüte von 0 bis 255. Größer ist besser. Beschreibt die Sauberkeit des Signals, nicht seine Lautstärke.

Kurzadresse

Vier Hexadezimalzeichen, mit denen ein Gerät in seinem Netz angesprochen wird, etwa `0x8f2a`. Kann sich beim Neuanmelden ändern.

IEEE-Adresse

Die feste, weltweit eindeutige Nummer eines Geräts (sechzehn Zeichen). Ändert sich nie.

Firmware

Die Software im Inneren des Sticks. Austauschbar — genau das tun wir in Kapitel 6.

Baud

Wie schnell zwei Geräte über eine Leitung reden. Beide müssen dieselbe Zahl benutzen, sonst kommt Unsinn an. Hier: 1 000 000.

14 Im Analyzer: einschalten, ablesen, benennen

Ab hier ist der Stick kein Werkzeug mehr, sondern Teil des Analyzers. Drei Handgriffe, alle über die Weboberfläche — **keine Konsole nötig**.

14.1 Einschalten

Die Reiter oben in der Oberfläche sind **zweizeilig**: oben steht, worum es geht, darunter, für welches Funknetz. Für Zigbee gibt es drei davon.



blau = die drei Zigbee-Reiter

Zweizeilige Reiter: oben die Sache, darunter das Funknetz.

Zum Einschalten: *Einstellungen · Zigbee* → „Zigbee einschalten ...“. Das wirkt **nach einem Neustart des Dienstes** — Ein- und Ausschalten heißt, die serielle Schnittstelle zu öffnen oder zu schließen, und das im laufenden Betrieb zu verdrahten wäre mehr Zustand als Nutzen. Ein Kanalwechsel dagegen wirkt sofort.

Reiter	erscheint
Einstellungen · Zigbee	immer . Hier sitzt der Schalter, mit dem Zigbee überhaupt erst eingeschaltet wird — hinge der Reiter selbst daran, käme man nach dem Ausschalten nicht mehr an ihn heran.
Meldungen · Zigbee	sobald der Mithörer dieses Analyzers eingeschaltet ist
Verbund · Zigbee	nur auf dem Master , und nur wenn er selbst einen Mithörer betreibt

WARUM DER MASTER SELBST MITHÖREN MUSS

Wer zusammenführt, soll selbst messen. Ein Master, der nur fremde Zahlen weiterreicht, hätte keine eigene Zeile in der Übersicht — und dann könnte niemand sagen, ob ein „nirgends gehört“ an den Standorten liegt oder daran, dass der Master gar nicht hinhört.

Auf den Clients darfst du Zigbee trotzdem betreiben, ganz unabhängig davon. Sie zeigen ihre eigenen Zahlen unter *Meldungen · Zigbee*. Ob sie in der gemeinsamen Übersicht auftauchen, entscheidet der Master über seine Gegenstellenliste — dieselbe wie bei BidCoS, keine zweite.

14.2 Was „Meldungen · Zigbee“ zeigt

Abschnitt	Antwortet auf
Zustand	Hört er? Auf welchem Kanal? Seit wann? Wie viele Pakete, wie viele verworfen?
Geräte	Wer sendet, wie stark, wie sauber — und was davon grenzwertig ist
Fremde Netze in Hörweite	Nachbarn auf demselben Kanal. Nicht deine Geräte, aber sie belegen deinen Empfang mit
Nicht gehört	Geräte aus deCONZ, die dieser Standort nie gehört hat — mit einem Standort noch kein Beweis für einen Defekt
Pakete	die laufende Liste der letzten zehn Minuten

Die Bewertung stützt sich auf den **Mittelwert**, nicht auf das schwächste je empfangene Paket. Wer eine Stunde misst, sieht bei jedem Gerät irgendwann einen Ausreißer; das sagt etwas über die Messdauer aus und nichts über das Gerät. Die Spanne steht daneben, entscheidet aber nicht.

14.3 Aus 0x837E wird „Deckenlampe Flur oben“

Der Mithörer sieht nur Kurzadressen. Deine Zigbee-Steuerung (deCONZ/Phoscon) kennt die Namen — aber zu **IEEE-Adressen**, und die Kurzadresse kennt sie nicht. Die Brücke baut der Analyzer selbst: Zigbee überträgt die IEEE-Adresse des Absenders im Paketkopf **unverschlüsselt** mit, in rund der Hälfte aller Datenrahmen.

Zwei Schritte, beide unter *Einstellungen · Zigbee*:

- 1 deCONZ-Rechner eintragen**
— IP oder Hostname, bei Bedarf mit `:Port`.
- 2 Schlüssel anfordern**
In Phoscon zuerst *Einstellungen* → *Gateway* → *Erweitert* → „App verbinden“ anklicken — das öffnet ein Fenster von 60 Sekunden. Dann hier auf **Schlüssel anfordern**.

WARUM ES KEIN EINGABEFELD FÜR DEN SCHLÜSSEL GIBT

deCONZ **zeigt bestehende Schlüssel nie an**. Es vergibt nur neue, und nur während dieses Fensters. Wer einen von Hand besorgt, trägt ein Zugangstoken durch Zwischenablage und Bildschirm.

Holt der Analyzer ihn selbst, sieht ihn niemand: Er steht in keiner Antwort, keinem Protokoll und keiner Fehlermeldung. Ein neuer Schlüssel widerruft den alten automatisch.

EINE FALLE BEI DER FEHLERSUCHE

Ob ein Schlüssel noch gilt, lässt sich **nicht** mit `GET /api/<schlüssel>/config` prüfen: Diesen Zweig beantwortet deCONZ **ohne jede Anmeldung**, auch für einen frei erfundenen Schlüssel. Nimm einen geschützten Zweig, etwa `/lights` — dort steht dann `unauthorized user`.

14.4 Was du davon hast

An einem einzigen Standort über 24 Stunden gemessen: **42 Geräte gehört, 34 davon mit Namen**, und aus der deCONZ-Liste blieben nur zwei übrig, die dieser Standort nie gehört hat. Statt einer Liste von Hexzahlen steht dann so etwas da:

Zwischenstecker Werkstatt	-61 dBm	LQI 255	gut
Deckenlampe Flur oben	-73 dBm	LQI 254	gut
Aussenleuchte Nord 02	-88 dBm	LQI 45	GRENZWERTIG
Aussenleuchte Nord 04	-89 dBm	LQI 6	GRENZWERTIG

(Erfundene Namen. Wie deine Geräte heißen, bestimmst du in deiner Zigbee-Steuerung — der Analyzer übernimmt sie nur.)

Zwei grenzwertige Geräte, beide an derselben Außenwand — das ist eine Aussage, mit der man etwas anfangen kann. Mit `0xD92F` und `0xE108` wäre es keine gewesen.

WARUM NICHT ALLE GERÄTE EINEN NAMEN BEKOMMEN

Die Brücke zum Namen ist die IEEE-Adresse, und die steht nur in einem Teil der Funkpakete. Ein Gerät, das gerade wenig sendet, kann deshalb eine Weile ohne Namen dastehen — es wird nachgetragen, sobald ein passendes Paket vorbeikommt. Der Koordinator selbst (Kurzadresse `0000`) bleibt namenlos: Er steht in der deCONZ-Geräteliste nicht, weil er sie führt.

15 Was noch fehlt

Der Weg vom Stick bis in die Oberfläche des Analyzers ist gebaut und läuft. Was noch fehlt, steht in der letzten Zeile dieser Aufstellung — und es ist weniger, als beim ersten Entwurf dieses Buches offen war:

Thema	Inhalt	Zustand
Fester Geräteiname	<code>/dev/asksin-zigbee</code> , nach jedem Neustart gleich (Kapitel 5)	fertig
Firmware aufspielen	durch den Analyzer selbst, im Setup und in der Weboberfläche (Kapitel 6.1)	fertig
Zigbee einschalten	im Einrichtungsassistenten und in der Weboberfläche	fertig
Die Zigbee-Seite	Zustand, Geräte, Pakete (Kapitel 14)	fertig
Gerätenamen	aus deCONZ, über die IEEE-Adresse (Kapitel 14.3)	fertig
Mehrere Standorte	Empfangsmatrix unter <i>Verbund · Zigbee</i> : welcher Analyzer hört welches Gerät am besten, und was hört niemand	fertig
Langzeitdaten	InfluxDB und eine eigene Grafana-Ansicht für den Mithörer	fertig
ioBroker	Zigbee-Werte als States im Adapter <code>asksin-analyzer-rpi</code>	fertig
Grundrauschen	gibt es für Zigbee bewusst noch nicht — die Firmware liefert RSSI nur je Paket, nicht zwischen den Paketen	offen

WAS ZWAR GEBAUT, ABER NOCH NICHT VOLLSTÄNDIG ERPROBT IST

Die **Empfangsmatrix** zeigt ihren eigentlichen Wert erst mit einem zweiten mithörenden Standort — bis dahin hat sie eine einzige Spalte. Und das **Aufspielen der Firmware durch den Analyzer** ist in allen Schritten geprüft *außer* dem Schreibvorgang selbst (siehe Kapitel 6.1). Beides hängt am zweiten Stick.

WIE DIESES BUCH GEPFLEGT WIRD

Solange die Erweiterung gebaut wird, lebt es hier für sich. Jede Angabe mit dem Vermerk **UNGEPRÜFT** wird ersetzt, sobald wir sie am Gerät gesehen haben. Erst wenn alles zufriedenstellend läuft, wandert der Inhalt als neues Kapitel ins große Handbuch.

Der zugehörige Arbeitsplan mit allen Entscheidungen und offenen Fragen liegt daneben: [projekt/zigbee-integration/README.md](#) , der laufende Verlauf in [stand.md](#) .